

# Pipelining

**Up till now:** we have designed a processor that execute all the SimpleRisc instructions.

Two styles:

Hardwired control unit

Microprogrammed control unit:

- microprogrammed datapath

- microassembly language

- microinstruction

# Designing efficient processors

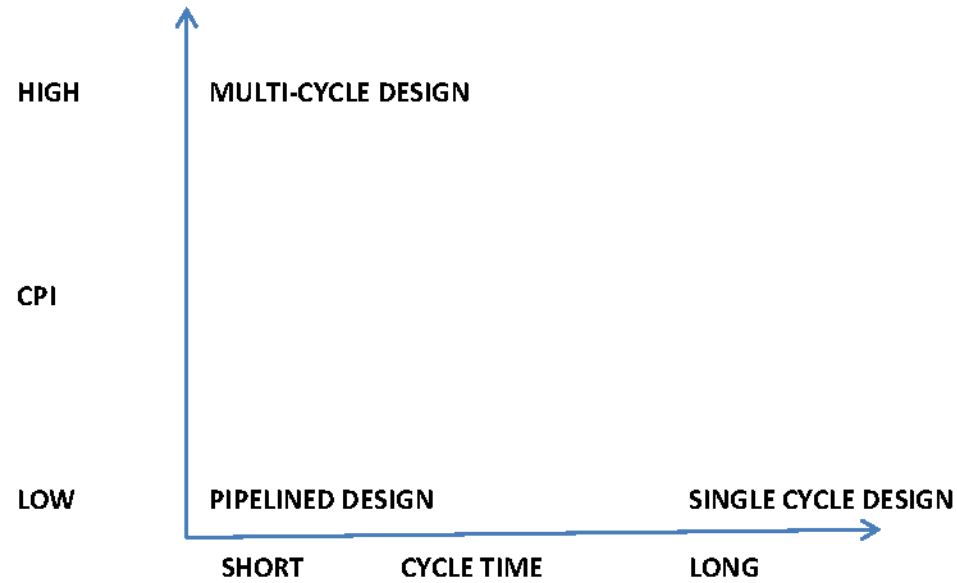
- \* **Microprogrammed** processors are much slower than **hardwired** processors
- \* Even **hardwired** processors
  - \* Have a lot of **waste** !!!
  - \* We have 5 stages.
  - \* What is the IF stage doing, when the MA stage is active ?
    - \* **ANSWER** : It is **idling**

# Resource utilization

- **Single cycle design:** each resource is tied up for the entire duration of the instruction execution.
- **Multi-cycle design:** resource utilized in cycle  $t$  of instruction  $I$  is available for cycle  $t+1$  of instruction  $I$ .
- **Pipelined design:** resource utilized in cycle  $t$  of instruction  $I$  is available again for cycle  $t$  of instruction  $I+1$ .

# Problems with single cycle design

- Slowest instruction pulls down the clock frequency
- Resource utilization is poor
- There are some instructions which are impossible to be implemented in this manner.

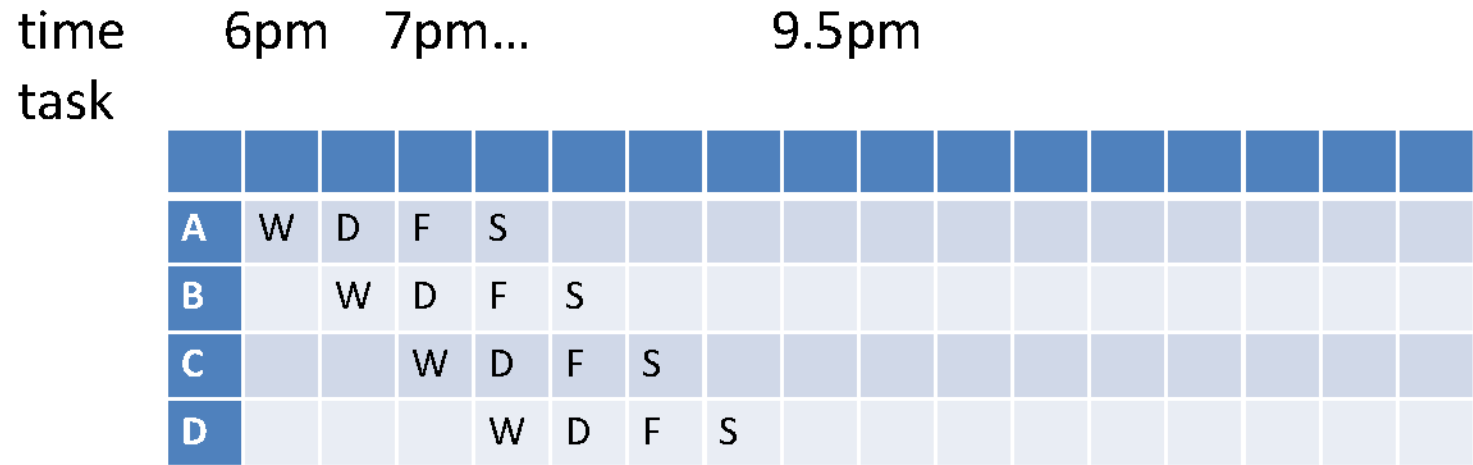


# The Notion of Pipelining

- \* Let us go back to the **car assembly line**
  - \* Is the **engine shop** idle, when the **paint shop** is painting a car ?
  - \* **NO** : It is building the engine of another car
  - \* When this engine goes to the body shop, it builds the engine of another car, and **so on ....**
- \* **Insight** :
  - \* **Multiple cars** are built at the same time.
  - \* A car **proceeds** from one stage to the next

- Pipelining is an implementation technique in which multiple instructions are overlapped in execution. Today pipelining is the key to making processors fast.
- Ex. Laundry analogy for pipelining: see Fig.

|      |     |          |   |   |   |   |   |   |   |   |   |   |   |   |   |   |     |
|------|-----|----------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|-----|
| time | 6pm | 7pm..... |   |   |   |   |   |   |   |   |   |   |   |   |   |   | 2am |
| task |     |          |   |   |   |   |   |   |   |   |   |   |   |   |   |   |     |
| A    | W   | D        | F | S |   |   |   |   |   |   |   |   |   |   |   |   |     |
| B    |     |          |   |   | W | D | F | S |   |   |   |   |   |   |   |   |     |
| C    |     |          |   |   |   |   |   |   | W | D | F | S |   |   |   |   |     |
| D    |     |          |   |   |   |   |   |   |   |   |   |   | W | D | F | S |     |



The washer ,dryer, folder and storer each take 30 minutes for their task. Sequential laundry takes 8 hrs for four loads of wash while pipelined laundry takes just 3.5 hrs

we show the pipeline stage of different loads overtime by showing copies of the four resources on this two dimensional time line, but we really have just one of each resource.

# Observed so far..

- The pipeline paradox is that the time from placing a single dirty sock in the washer until it is dried, folded and put away is not shorter for pipelining.
- The reason pipelining is faster for many loads is that everything is working in parallel, so more loads are finished per hour,
- Pipelining improves throughput of the laundry system without improving the time to complete a single load.
- Hence, pipelining would not decrease the time to complete one load of laundry, but when we have many loads of laundry to do, the improvement in throughput decreases the total time to complete the work.



- If all the stages take about the same amount of time and there is enough work to do, then the speedup due to pipelining is equal to the number of stages in the pipeline (in this case four stages).
- 20 loads would take about 5 times as long as 1 load, while 20 loads of sequential laundry takes 20 times as long as 1 load.
- It's only  $8/3.5 = 2.3$  times faster because only four loads are shown.
- The beginning and end of the workload in the pipelined version, the pipeline is not completely full.
- This start-up and wind-down affects performance when the number of tasks is not large as compared to the number of stages in the pipeline, if the number of loads is much larger than 4, then the stages will be full most of the time and the increase in throughput will be very close to 4.

- The same principles apply to processors, where we pipeline instruction execution. MIPS instructions classically takes five steps:
  - 1)Fetch instruction from memory
  - 2)Read register while decoding the instruction(The format of MIPS allows reading and decoding to occur simultaneously).
  - 3)Execute the operation or calculate the address.
  - 4)Access an operand in data memory.
  - 5)Write the result into a register.

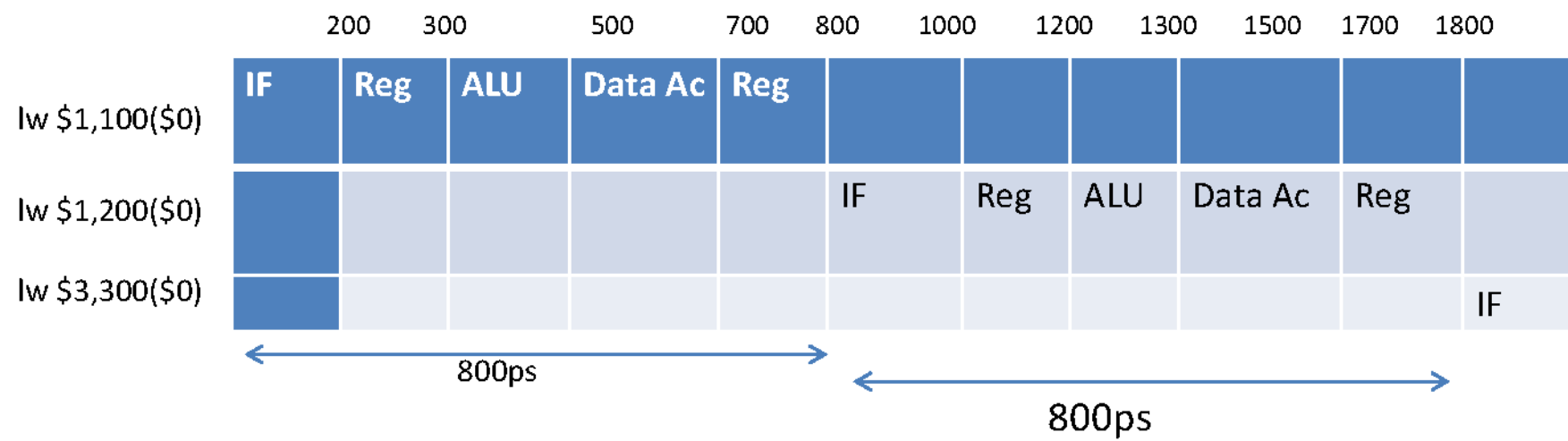
# Single cycle Vs pipelined performance

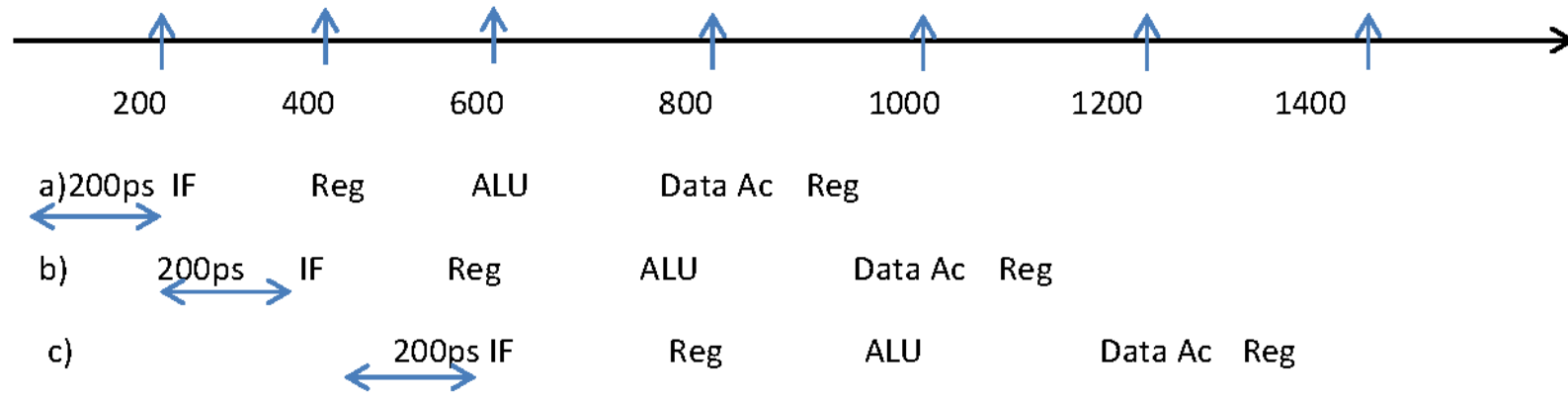
Compare the avg time between instructions of a single cycle implementation, in which all instructions take one clock cycle to a pipelined implementation.

The operation times for major functional units as example is:

(in the single cycle model every instruction takes exactly one clock cycle,so the clock cycle must be stretched to accommodate the lowest instruction.)

| Inst class | Inst fetch | Reg read | ALU oper | Data access | Reg write | total |
|------------|------------|----------|----------|-------------|-----------|-------|
| Load       | 200ps      | 100ps    | 200ps    | 200ps       | 100ps     | 800ps |
| Store      | 200ps      | 100ps    | 200ps    | 200ps       |           | 700ps |
| R-format   | 200ps      | 100ps    | 200ps    |             | 100ps     | 600ps |
| branch     | 200ps      | 100ps    | 200ps    |             |           | 500ps |





a)lw \$1,100(\$0)

b)lw \$2,200(\$0)

c)lw \$3,300(\$0)

- Avg time between the instructions reduced from 800ps to 200 ps
- Computer pipeline stage times are limited by the slowest resource. Either the ALU operation or the memory access.
- We assume that the write to register file occurs in the first half of the clock cycle and the read from the register file occurs in the second half of the clock cycle.
- For speedup:

time between instructions(pipelined)=time between  
instruction(nonpipelined)/number of pipe stages

Improvement is  $800/5=160$ ps clock cycle.(stages may be imperfectly balanced or some overheads).

Thus the time per instruction in the pipelined processor will exceed the minimum possible and speedup will be less than the number of pipeline stages.

- Our claim of fourfold improvement is not reflected in the total execution of three instructions( $2400/1400=1.8$ ).this is becoz the number of instructions is not large.Let us increase the number of instructions:
- Let us add 1000,000 instructions in the pipeline,each instruction adds 200ps ,to the total execution time.
- Total execution time= $1000,000*200ps+1400ps=200,001400ps$
- For nonpipelined: $1000,000*800+2400=800,002400ps$
- Ratio= $800,002400/200,001400=4=800/200$
- *Note:pipeline improves performance by increasing instruction throughput,as opposed to decreasing the execution time on an individual instruction.*

# Pipeline hazards

- There are situations in the pipeline when the next instruction can not execute in the following clock cycle, these events are called hazards and there are three types:
- Structural, data and control hazards.
- **Structural Hazard**: An occurrence in which planned instruction can not execute in the proper clock cycle because the hardware cannot support the combinations of instructions that are set to execute in the given clock cycle.
- With reference to laundry example the structural hazard would occur if we use the combination of washer-dryer instead of separate washer and dryer, or you roommate was busy doing something else and wouldn't put clothes away.
- Hence, the carefully scheduled pipeline plans would than be foiled.
- If the pipeline in the previous Fig, has fourth instruction then we would see that the first instruction is accessing data from the memory and the fourth instruction is fetching an instruction from that same memory.
- Without two memories our pipeline could have a structural hazard.



- \* A **structural hazard** may occur when two instructions have a conflict on the same set of resources in a cycle
  - \* Example :
    - \* Assume that we have an add instruction that can read one operand from memory
    - \* `add r1, r2, 10[r3]`
- ```
[1]: st r4, 20[r5]
[2]: sub r8, r9, r10
[3]: add r1, r2, 10[r3]
```
- \* This code will have a structural hazard
    - \* [3] tries to read 10[r3] (MA unit) in cycle 4
    - \* [1] tries to write to 20[r5] (MA unit) in cycle 4
  - \* Does not happen in our pipeline

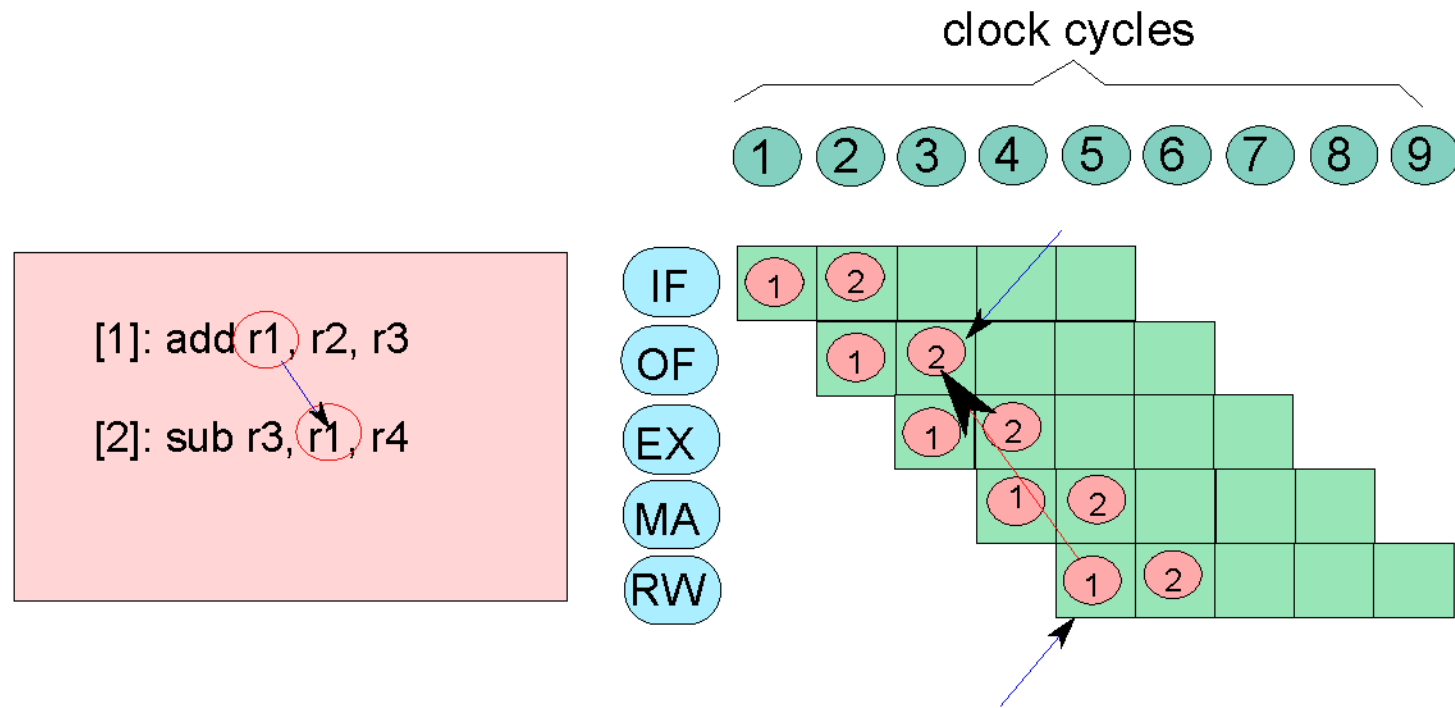
- **Data Hazard:** An occurrence in which a planned instruction can not execute in the proper clock cycle because the data that is needed to execute the instruction is not yet available.
- Occurs when the pipeline must be stalled because one step must wait for another to complete.
- In the computer pipeline, data hazards arise from the dependence of one instruction on an earlier one that is still in the pipeline (relationship that doesn't really exist when doing laundry).

add \$s0,\$t0,\$t1

sub \$t2,\$s0,\$t3

Data hazard could severely stall the pipeline, the add instruction does not write its result until the fifth stage, meaning that we would have to add three bubbles to the pipeline.

# Data Hazard



\* Instruction 2 will read incorrect values !!!

### Definition 61

*A hazard is defined as the possibility of erroneous execution of an instruction in a pipeline. A data hazard represents the possibility of erroneous execution because of the unavailability, or availability of incorrect data.*

- \*This situation represents a **data hazard**
- \*In specific,
  - \*it is a **RAW** (read after write) hazard
- \*The earliest we can dispatch instruction 2, is cycle 5